

```

1 import components.simplereader.SimpleReader;
2
3 /**
4  * Program to evaluate XMLTree expressions of {@code int}.
5  *
6  * @author Eric Zhou
7  */
8
9 public final class XMLTreeIntExpressionEvaluator {
10
11     /**
12      * Private constructor so this utility class cannot be instantiated.
13      */
14     private XMLTreeIntExpressionEvaluator() {
15     }
16
17     /**
18      * Evaluate the given expression.
19      *
20      * @param exp
21      *         the {@code XMLTree} representing the expression
22      * @return the value of the expression
23      * @requires <pre>
24      * [exp is a subtree of a well-formed XML arithmetic expression] and
25      * [the label of the root of exp is not "expression"]
26      * </pre>
27      * @ensures evaluate = [the value of the expression]
28      */
29     private static int evaluate(XMLTree exp) {
30         assert exp != null : "Violation of: exp is not null";
31
32         int result = 0;
33         if (exp.label().equals("number")) { //decide the end of recursion
34             result = Integer.parseInt(exp.attributeValue("value"));
35         } else {
36             int left = evaluate(exp.child(0)); //recursion
37             int right = evaluate(exp.child(1));
38             String op = exp.label(); // check the operator
39
40             if (op.equals("plus")) {
41                 result = left + right;
42             } else if (op.equals("minus")) {
43                 result = left - right;
44             } else if (op.equals("times")) {
45                 result = left * right;
46             } else if (op.equals("divide")) {
47                 if (right == 0) { // assert that cannot divide zero
48                     Reporter.fatalErrorToConsole("cannot divide by zero");
49                 }
50                 result = left / right;
51             }
52         }
53         return result;
54     }
55
56     /**
57      * Main method.
58      *
59      * @param args
60      *         the command line arguments
61      */
62     public static void main(String[] args) {
63         SimpleReader in = new SimpleReader11();
64         SimpleWriter out = new SimpleWriter11();
65     }
66 }

```

```
72 out.print("Enter the name of an expression XML file: ");
73 String file = in.nextLine();
74 while (!file.equals("")) {
75     XMLTree exp = new XMLTree1(file);
76     out.println(evaluate(exp.child(0)));
77     out.print("Enter the name of an expression XML file: ");
78     file = in.nextLine();
79 }
80
81 in.close();
82 out.close();
83 }
84
85 }
86
```